UNIVERSIDAD POPULAR DEL CESAR

Facultad de Ingeniería y Tecnologías

Ingeniería de Sistemas e Informática

GUÍA DEL TALLER

Refactorización a Código Limpio

Sistema de Facturación — Restaurante El Buen Sabor

Programación de Computadores III SS462 Cuarto Semestre

Docente: Ing. Alfredo David Bautista Romero

Estudiante: **Andres Felipe Zabaleta Díaz**

Universidad Popular del Cesar 2026

1. ¿Qué van a encontrar?

El proyecto RestauranteBuenSabor es un sistema de facturación para un restaurante que funciona correctamente, pero está escrito con el máximo de malas prácticas posibles. La misión es refactorizarlo hasta convertirlo en código limpio, sin alterar el comportamiento del programa.

> ⚠ **El programa produce resultados correctos. El problema no es que falle — el problema es que es imposible de mantener, entender y modificar. Eso es exactamente lo que pasa en la industria real.**

1.1 Estructura de archivos entregada

```
RestauranteBuenSabor/
├── src/
│   └── restaurante/
│       ├── RestauranteElBuenSabor.java  — punto de entrada y menú interactivo
│       ├── Datos.java                   — almacén global de datos del sistema
│       ├── Proceso.java                 — lógica de cálculo de la factura
│       ├── Utilidades.java              — métodos auxiliares
│       └── Imprimir.java                — todo lo que se imprime en pantalla
└── README.md                            — instrucciones para compilar y ejecutar
```

2. ¿Qué hace el sistema?

El programa simula la operación de caja de un restaurante. Al ejecutarlo aparece un menú con 6 opciones que se repite en bucle hasta que el usuario elige salir:

```
========================================
    RESTAURANTE EL BUEN SABOR
    Calle 15 #8-32, Valledupar
    NIT: 900.123.456-7
========================================
1. Ver carta
2. Agregar producto al pedido
3. Ver pedido actual
4. Generar factura
5. Nueva mesa
0. Salir
========================================
Seleccione una opcion:
```

2.1 Carta del restaurante (Opción 1)

#	Producto	Precio
1	Bandeja Paisa	$32.000
2	Sancocho de Gallina	$28.000
3	Arepa con Huevo	$8.000
4	Jugo Natural	$7.000
5	Gaseosa	$4.500
6	Cerveza Poker	$6.000

#	Producto	Precio
7	Agua Panela	$3.500
8	Arroz con Pollo	$25.000

2.2 Reglas de negocio (Opción 4 — Generar factura)

- Subtotal: suma de precio × cantidad para cada producto pedido.
- Descuento 5 %: se aplica si hay más de 3 productos diferentes en el pedido.
- IVA 19 %: se calcula sobre el subtotal después del descuento.
- Propina 10 %: se aplica si el subtotal (antes del IVA) supera $50.000.
- Total: subtotal con descuento + IVA + propina.

Ejemplo de factura generada (Bandeja Paisa x2 + Jugo Natural x2):

```
FACTURA No. 001
----------------------------------------
Bandeja Paisa        x2      $64.000
Jugo Natural         x2      $14.000
----------------------------------------
Subtotal:                   $78.000
IVA (19%):                  $14.820
Propina (10%):              $ 9.282
----------------------------------------
TOTAL:                      $102.102
```

3. Catálogo de Malas Prácticas

A continuación, se documenta cada mala práctica presente en el proyecto, con su archivo, número de línea exacto y descripción. La columna Dónde incluye el archivo y las líneas concretas del código fuente para facilitar la localización.

> ⚠ **Total: 34 malas prácticas identificadas y documentadas en el proyecto.**

3.1 Datos.java — El God Object

Almacena todos los datos del sistema en variables estáticas públicas. Cualquier clase puede leer y modificar cualquier dato directamente, sin control ni encapsulamiento.

#	Problema	Dónde – Archivo y Línea	Descripción
1	Arrays paralelos	Datos.java · Líneas 12-14	nom[], p[] y cant[] son tres arrays independientes ligados por índice. Si se cambia el orden de uno, los otros se desincronzan. Solución: clase Producto(nombre, precio, cantidad).
2	Todo public static	Datos.java · Líneas 12-18	Todas las variables son public static: cualquier clase las modifica directamente, sin getters ni setters. Solución: hacerlas private con métodos de acceso controlado.
3	Nombre hardcodeado (1ª vez)	Datos.java · Línea 17	String x="El Buen Sabor" usa una variable de un solo carácter y repite el nombre del restaurante. Solución: constante pública NOMBRE_RESTAURANTE en una clase de configuración.
4	Variable tmp sin propósito	Datos.java · Línea 18	public static String tmp="" jamás cumple una función significativa; su propósito es ambiguo. Solución: eliminarla por completo.
5	Sin constructor ni métodos	Datos.java · Clase completa	La clase es un simple contenedor de campos públicos sin ningún comportamiento. Solución: agregar constructor que inicialice los datos y métodos de acceso.
6	Nombres de 1-3 letras	Datos.java · Líneas 15-17 (ms, est, tot, nf, p, x)	ms=mesa, est=estado, tot=total, nf=núm. factura, p=precios, x=nombre — todos ilegibles sin contexto. Solución: nombresMesa, estadoMesa, totalFactura, numeroFactura, precios, nombreRestaurante.

3.2 Proceso.java — El método que hace todo

Contiene la lógica de cálculo del total de la factura en un único método de más de 80 líneas.

#	Problema	Dónde – Archivo y Línea	Descripción
7	hacerTodo() con múltiples responsabilidades	Proceso.java · Líneas 12-46	Un único método calcula el subtotal, aplica descuento, calcula IVA, aplica propina y actualiza el estado global. Solución: extraer calcularSubtotal(), aplicarDescuento(), calcularIva(), aplicarPropina().
8	IVA duplicado 3 veces	Proceso.java · Líneas 24, 30 y 38/43	La expresión iva = variable * 0.19 aparece literalmente tres veces en el mismo método. Solución: método calcularIva(double base) o constante TASA_IVA = 0.19.

#	Problema	Dónde – Archivo y Línea	Descripción
9	Magic numbers sin nombre	Proceso.java · Líneas 22, 24-27, 38-43 (0.05, 0.19, 0.10, 50000, 3)	Todos los valores del negocio (tasas, umbrales, mínimos) están escritos como literales. Solución: PORCENTAJE_DESCUENTO, TASA_IVA, PORCENTAJE_PROPINA, UMBRAL_PROPINA, MIN_PRODUCTOS_DESCUENTO.
10	Condicionales anidados 4 niveles	Proceso.java · Líneas 20-44	if(cont>3){ if(sub>0){ if(aux>50000){...}else{...} } } genera un código en pirámide ilegible. Solución: guard clauses y extracción de métodos para reducir el anidamiento.
11	Código comentado 'versión anterior'	Proceso.java · Líneas 32-35	Tres líneas de lógica antigua con '// version anterior - no borrar'. Git guarda el historial. Solución: eliminar el código muerto.
12	procesar() con 7 parámetros	Proceso.java · Línea 47 (a, b, c, d, e, f, g)	Los 7 parámetros de una sola letra son imposibles de interpretar en la llamada. Solución: objeto ParametrosCalculo o reducir las responsabilidades del método.
13	Variable aux innecesaria	Proceso.java · Línea 55-56	tmp=iva; res=res+tmp; — tmp es un intermediario sin valor; podría ser directamente res+=iva. Solución: eliminar tmp y operar sobre iva directamente.
14	Comentarios que repiten el código	Proceso.java · Líneas 16, 25, 29, 39, 49, 52, 54, 58	'// multiplica precio por cantidad' junto a sub+= precio*cant repite lo obvio. Solución: si se necesita explicar, mejorar los nombres; eliminar los comentarios redundantes.
15	Comentarios de cierre inútiles	Proceso.java · Líneas 19, 31, 44	// fin while, // fin if sub>0, // fin if-else cont señalan que el método es demasiado largo. Solución: extraer bloques en métodos con nombres expresivos.

3.3 Utilidades.java — Nombres engañosos

Contiene métodos auxiliares: calcular, validar y reiniciar el pedido.

#	Problema	Dónde – Archivo y Línea	Descripción
16	calcular() con 7 parámetros	Utilidades.java · Línea 12 (pr, cn, dc, iv, pp, ni, ap)	Duplica exactamente la lógica de Proceso.procesar() con 7 parámetros igualmente crípticos. Solución: eliminar el duplicado y centralizar el cálculo en un solo lugar.
17	validar() con efecto secundario oculto	Utilidades.java · Líneas 23-30	validar() también reinicia Datos.tot y Datos.tmp si el pedido está vacío, sin que la firma lo indique. Solución: separar en validarPedido() y reiniciarTotales().
18	Nombre hardcodeado (3ª vez)	Utilidades.java · Línea 20	System.out.println("RESTAURANTE EL BUEN SABOR - calculo aplicado") dentro de un método de cálculo mezcla presentación con lógica. Solución: eliminar el println del método.
19	Bloque de ~20 líneas de código comentado	Utilidades.java · Líneas 32-47	El método reiniciar() contiene calcOld() completo y varios bloques comentados. Código muerto que confunde. Solución: eliminar todo; git conserva el historial.
20	Variables tmp y aux2 redundantes	Utilidades.java · Líneas 13, 17 y 21	tmp guarda res*iv y aux2 copia res solo para retornarla. Ambas son intermediarios sin valor. Solución: res += res * iv; return res;

3.4 Imprimir.java — Código copiado y pegado

Contiene todos los métodos de impresión en consola.

#	Problema	Dónde – Archivo y Línea	Descripción
21	imprimirFacturaCompleta() con 3 responsabilidades	Imprimir.java · Líneas 34-78	Recalcula totales, imprime la factura y actualiza el estado (Datos.nf++, Datos.est). Solución: calcularTotales(), imprimirFactura() y actualizarEstado() por separado.
22	Cálculo del total duplicado	Imprimir.java · Líneas 34-50 y 79-95	imprimirFacturaResumen() contiene el mismo bloque de cálculo que imprimirFacturaCompleta(). Cambiar una regla obliga a actualizar los dos. Solución: método calcularTotales() compartido.
23	80 % del código idéntico entre métodos	Imprimir.java · Líneas 34-78 vs 79-110	Ambos métodos comparten encabezado del restaurante, separador, cálculos y formato printf. Solución: imprimirEncabezado(), imprimirSeparador() y calcularTotales() reutilizables.
24	Nombre hardcodeado (4ª vez)	Imprimir.java · Líneas 14, 53-55, 72-73, 98-100	"RESTAURANTE EL BUEN SABOR" está escrito como literal en 4 lugares distintos dentro del mismo archivo. Solución: constante NOMBRE_RESTAURANTE.
25	Formato printf repetido sin constante	Imprimir.java · Líneas 19, 28, 33, 62, 65-68, 70, 104-109	La cadena "%-27s $%,.0f%n" se repite 7 veces. Solución: constante FORMAT_LINEA y método imprimirLinea(String, double).

3.5 RestauranteElBuenSabor.java — El caos central

Punto de entrada del programa. El método main mezcla presentación, validación y lógica de negocio en más de 100 líneas.

#	Problema	Dónde – Archivo y Línea	Descripción
26	Método main de 100+ líneas	RestauranteElBuenSabor.java · Líneas 14-132	El main mezcla menú, validación de entradas, lógica de mesa y control de flujo en 118 líneas. Solución: mostrarMenu(), procesarOpcion(), agregarProducto(), gestionarMesa().
27	Dos Scanner para el mismo System.in	RestauranteElBuenSabor.java · Líneas 15 y 123	Scanner sc se crea al inicio y Scanner sc2 dentro del bloque de opción inválida. Dos Scanner sobre el mismo InputStream generan comportamiento impredecible. Solución: un único Scanner global.
28	Variables sin uso	RestauranteElBuenSabor.java · Línea 16 (aux, tmp, m, continuar, x)	aux (String), tmp (int), m (double), continuar (boolean) y x (int) se declaran pero su uso es mínimo o nulo; continuar siempre se asigna true sin efecto. Solución: eliminarlas.
29	Lógica de negocio en el main	RestauranteElBuenSabor.java · Líneas 44-63	Validación del número de mesa, gestión del estado est y cálculo de variables auxiliares viven en el punto de entrada. Solución: mover a clases de servicio.
30	Nombre hardcodeado (5ª vez)	RestauranteElBuenSabor.java · Líneas 18-20	"RESTAURANTE EL BUEN SABOR" se imprime en el main como literal. Quinta y última ocurrencia. Solución: constante NOMBRE_RESTAURANTE y método imprimirEncabezado().

3.6 Problemas de formato visual (todos los archivos)

#	Problema	Dónde – Archivo y Línea	Descripción
31	Sin indentación	Todos los archivos (todas las líneas)	El código está pegado al margen izquierdo independientemente del nivel de anidamiento. Solución: Source > Format All en NetBeans.
32	Llaves inconsistentes	Todos los archivos (mezcla K&R / Allman)	Algunas llaves abren en la misma línea y otras en la siguiente sin criterio uniforme. Solución: escoger un estilo y aplicarlo con el formateador del IDE.
33	Llaves de cierre apiladas	Proceso.java · Líneas 31, 44; Imprimir.java · Líneas 50, 63, 78	}}// fin while o }// fin if-else apilan cierres de bloque en una sola línea. Solución: cada llave de cierre en su propia línea con la indentación del nivel correspondiente.
34	Múltiples sentencias por línea	Datos.java · Línea 15-16; Proceso.java · Líneas 13, 45; Utilidades.java · Líneas 13, 17, 48-49	public static int ms=0;public static int est=0; pone dos declaraciones en una línea. Solución: una sentencia por línea.

Referencias

Robert C. Martin — Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2009.

Shahan Chowdhury — The Clean Code Handbook. FreeCodeCamp, Enero 2025.